

Lab 3: Search and A*

Using an LLM as an Engineering Assistant

Artificial Intelligence Laboratory

1. Objective

This laboratory focuses on formulating a warehouse robot navigation problem as a search problem and implementing the A* search algorithm with the assistance of a Large Language Model (LLM).

The laboratory covers:

- Formulating a navigation problem as a search problem
 - Identifying states, actions, transitions, initial state, goal state, and costs
 - Designing a search agent before implementation
 - Using an LLM to assist in implementing A* search
 - Testing and validating the generated program
 - Comparing BFS and A*
 - Investigating the effect of different heuristic functions
 - Evaluating the use of an LLM as an engineering assistant
-

2. Problem Description

The warehouse robot navigation problem is represented using a two-dimensional grid.

The symbols have the following meanings:

- S — starting position
- G — goal position
- # — obstacle
- . — free cell

The robot can move one cell at a time in four directions:

- Up
- Down
- Left
- Right

Every movement has a cost of 1.

The objective is to find a valid path from the starting position s to the goal position g while avoiding obstacles.

The warehouse used in the experiment is:

```
#####
#S....#.....#
#.###.#.#####.#
#...#.#.....#.#
###.#.#####.#.#
#...#.....#.#
#.#.#####.#.#
#.....#G#
#####
```

3. Task 0: Search Problem Formulation

The warehouse navigation problem can be formulated as a search problem using states, actions, transitions, an initial state, a goal state, and a cost function.

Component	Definition
State	Current robot position represented as <code>(row, column)</code>
Actions	Up, Down, Left, Right
Transition	Move to an adjacent valid cell
Initial State	Position of s
Goal State	Position of g
Cost	1 for every movement

State Information

A state contains the current row and column of the robot.

For example:

```
(row, column)
```

The starting state in the original warehouse is:

```
(1, 1)
```

and the goal state is:

```
(7, 15)
```

Invalid Actions

An action is invalid if the resulting position:

- lies outside the grid, or
- contains an obstacle #.

Determinism

The environment is deterministic because a valid action always results in the same next state.

Solution

A solution is a sequence of valid movements that takes the robot from s to g without passing through obstacles.

4. Task 1: Design the Search Agent

Before requesting code from the LLM, the search agent was designed conceptually.

State Representation

Each state is represented as a tuple:

```
(row, column)
```

Warehouse Representation

The warehouse is represented as a two-dimensional Python list.

Valid Actions

For each state, the agent checks the four possible movements and accepts a movement only if the resulting cell is inside the grid and is not an obstacle.

Goal Recognition

The goal is reached when:

```
current state == goal state
```

Frontier

The A* frontier stores states according to their priority:

$$f(n) = g(n) + h(n)$$

where:

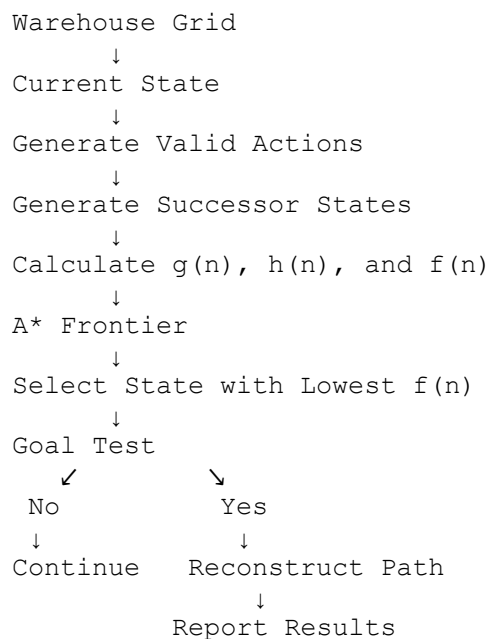
- $g(n)$ is the cost from the start to the current state.

- $h(n)$ is the estimated cost from the current state to the goal.
- $f(n)$ is the total estimated cost.

Path Reconstruction

A parent map stores the state from which each state was reached. Once the goal is reached, the path can be reconstructed by following the parent references backwards from the goal to the start.

Agent Design



The search agent was designed before asking the LLM to generate the implementation.

5. Task 2: LLM-Assisted A* Implementation

An LLM was used as an engineering assistant to generate the Python implementation of A*.

The prompt specified:

- the complete warehouse grid;
- the meaning of each symbol;
- the four possible movements;
- unit movement cost;
- state representation;
- valid-action checking;
- Manhattan distance as the heuristic;

- the A* priority function;
- path reconstruction;
- prevention of unnecessary repeated expansion;
- required output;
- use of only the Python standard library.

The LLM-generated implementation used Python's `heapq` module to implement the frontier as a priority queue.

The main components of the implementation were:

- `warehouse` — stores the grid;
- `heuristic()` — calculates Manhattan distance;
- `get_valid_actions()` — generates valid neighbouring states;
- `g_cost` — stores the best known cost from the start;
- `parent` — stores predecessor information;
- `expanded_states` — keeps track of expanded states;
- `reconstruct_path()` — reconstructs the final path;
- `a_star()` — performs the A* search.

The generated code was treated as a proposed implementation and was tested rather than assumed to be correct.

6. Task 3: Testing the A* Agent

The generated A* program was tested using four different cases.

Test 1: Original Warehouse

Result

```
Solution found: Yes
Path length: 40
States expanded: 64
```

The discovered path was:

```
[(1, 1), (1, 2), (1, 3), (1, 4), (1, 5),
 (2, 5), (3, 5), (4, 5), (5, 5),
 (5, 6), (5, 7), (5, 8), (5, 9), (5, 10),
 (5, 11), (5, 12), (5, 13),
 (4, 13), (3, 13), (3, 12), (3, 11),
 (3, 10), (3, 9), (3, 8), (3, 7),
 (2, 7), (1, 7), (1, 8), (1, 9), (1, 10),
 (1, 11), (1, 12), (1, 13), (1, 14), (1, 15),
 (2, 15), (3, 15), (4, 15), (5, 15),
 (6, 15), (7, 15)]
```

The path contains 41 positions, corresponding to 40 movements.

The resulting warehouse visualization was:

```
#####  
#S***#*****#  
#.#**#*#*#####*#  
#...#*#*#*#*#*#*#  
###.#*#####*#*#  
#...#*#*#*#*#*#*#  
#.#*#####.#*#  
#.....#G#  
#####
```

The test successfully demonstrated that the agent could navigate the original warehouse from S to G.

Test 2: One-Step Solution

The test map was:

```
#####  
#SG##  
#####
```

Result

```
Solution found: Yes  
Solution path: [(1, 1), (1, 2)]  
Path length: 1  
States expanded: 2
```

The agent correctly identified the one-step solution.

Test 3: No-Solution Map

The test map was:

```
#####  
#S....#  
###.###  
#...#G#  
#####
```

Result

```
Solution found: No  
Path length: N/A  
States expanded: 9
```

The search terminated normally and reported that no path existed. It did not enter an infinite loop.

Test 4: Alternative Paths

The test map was:

```
#####  
#S...G#  
#.....#  
#####
```

Result

```
Solution found: Yes  
Solution path:  
[(1, 1), (1, 2), (1, 3), (1, 4), (1, 5)]  
  
Path length: 4  
States expanded: 5
```

The direct path has a length of 4, which is the minimum possible distance between the start and goal in this map.

7. Task 4: Inspect the A* Algorithm

The major concepts from the search formulation can be mapped directly to the generated code.

Concept	Implementation
State	(row, column) tuple
Action	DIRECTIONS
Transition	get_valid_actions()
Goal Test	current == goal
$g(n)$	g_cost and $new_g = current_g + 1$
$h(n)$	heuristic() using Manhattan distance
$f(n)$	$new_f = new_g + heuristic(neighbour)$
Frontier	heapq priority queue
Expanded States	expanded_states
Path Reconstruction	reconstruct_path()

Frontier

The frontier is implemented using Python's `heapq` priority queue.

Each entry contains:

```
(f, g, state)
```

The state with the smallest `f` value is selected next using `heapq.heappop()`.

Heuristic

The Manhattan heuristic is calculated using:

```
h(n) = |row - goal_row| + |column - goal_column|
```

Cost Function

The cost of moving to a neighbouring state is:

```
new_g = current_g + 1
```

A* Evaluation Function

The implementation explicitly calculates:

```
f(n) = g(n) + h(n)
```

Preventing Repeated Expansion

The algorithm uses `expanded_states` to avoid expanding the same state repeatedly.

It also checks whether the `g` value stored in the priority queue is still equal to the best known `g_cost`.

Path Reconstruction

The `parent` dictionary stores the predecessor of every discovered state.

Starting from the goal, the algorithm repeatedly follows the parent references until reaching the start state. The resulting sequence is then reversed to obtain the path from `s` to `G`.

8. Task 5: BFS vs A* Comparison

BFS and A* were run on the same original warehouse.

Measure	BFS	A*
Solution found	Yes	Yes
Path length	40	40
States expanded	64	64

Observations

Both BFS and A* successfully found a solution.

Both algorithms produced a path of length 40.

Both algorithms expanded 64 states in this particular experiment.

Therefore, A* did not expand fewer states than BFS for this warehouse.

A* uses heuristic information to estimate the remaining distance to the goal, whereas BFS explores states based on their distance from the start. The benefit of A* depends on the structure of the search problem and the heuristic. For this particular warehouse, the measured number of expanded states was the same.

9. Task 6: Heuristic Investigation

Four different heuristic functions were tested on the same warehouse.

Heuristic	Solution Found	Path Length	States Expanded
Manhattan distance	Yes	40	64
$h(n) = 0$	Yes	40	64
Euclidean distance	Yes	40	64
$2 \times \text{Manhattan}$	Yes	40	64

Manhattan Distance

The Manhattan heuristic is:

$$h(n) = |\text{row} - \text{goal_row}| + |\text{column} - \text{goal_column}|$$

It is appropriate for this warehouse because the robot can move only horizontally or vertically and each movement has cost 1.

$$h(n) = 0$$

When:

$$h(n) = 0$$

A* uses only the actual cost from the start:

$$f(n) = g(n)$$

Thus, it provides no additional goal-directed heuristic information.

Euclidean Distance

The Euclidean heuristic is:

$$h(n) = \sqrt{(\text{row} - \text{goal_row})^2 + (\text{column} - \text{goal_column})^2}$$

It estimates the straight-line distance between the current state and the goal.

2 × Manhattan

The more aggressive heuristic is:

$$h(n) = 2 \times \text{Manhattan}(n)$$

It multiplies the Manhattan estimate by 2.

Observations

All four heuristics produced:

```
Path length = 40  
States expanded = 64
```

for this particular warehouse and implementation.

Therefore, no difference was observed in the measured results.

However, the heuristics have different theoretical properties. Manhattan distance is admissible for this four-directional unit-cost grid because it does not overestimate the actual remaining cost.

The 2 × Manhattan heuristic can overestimate the true remaining cost and is therefore not generally admissible.

Although 2 × Manhattan still produced the optimal 40-move path in this experiment, this single experiment does not establish that the heuristic will always preserve optimality.

10. Task 7: Evaluation of the LLM-Generated Agent

What did the LLM get correct?

The LLM correctly implemented the major components required for the A* search:

- State representation using `(row, column)`
- Four-directional movement
- Valid successor generation
- Manhattan distance
- Priority-queue frontier
- $g(n)$ cost tracking
- $f(n) = g(n) + h(n)$
- Goal detection
- Parent tracking
- Path reconstruction
- Prevention of repeated expansion
- Reporting of path length and states expanded

Bugs or Design Issues

No major functional bug was found during the tests performed.

However, one design limitation is that the warehouse, start, and goal are stored as global variables. This works for the laboratory experiments but would make the implementation less reusable in a larger program.

Another detail is that the path-printing function modifies the warehouse by replacing path cells with `*`. Therefore, the original warehouse needs to be restored before another experiment is run on the same map.

What was learned from inspecting the generated code?

Inspecting the generated code helped connect the theoretical A* concepts with their implementation.

In particular:

- `g_cost` represents the cost from the start.
- `heuristic()` calculates $h(n)$.
- The priority queue implements the frontier.
- `f(n)` determines the priority of a state.
- `expanded_states` prevents repeated expansion.

- `parent` enables path reconstruction.

What was tested?

The implementation was tested using:

1. The original warehouse
2. A one-step solution
3. A no-solution warehouse
4. A warehouse with alternative paths
5. BFS vs A*
6. Four different heuristic functions

These tests checked both successful and unsuccessful search cases.

Would the LLM-generated code be trusted without testing?

No.

LLM-generated code should be treated as a proposed implementation rather than automatically correct software. Testing is necessary because generated code may contain bugs, incorrect assumptions, or design choices that do not handle particular cases.

The experiments demonstrated the importance of validating the generated implementation against the actual problem requirements.

11. Final Reflection

Why is problem formulation important before writing code?

Problem formulation is important because it clearly defines the states, actions, transitions, initial state, goal state, and costs of the problem. This provides a precise specification for the search agent before implementation begins.

In this laboratory, representing the robot's position as a state and the four possible movements as actions made the warehouse navigation problem suitable for search algorithms.

Why is A* considered an informed search algorithm?

A* is an informed search algorithm because it uses a heuristic function to estimate the remaining cost from the current state to the goal.

It combines:

$$f(n) = g(n) + h(n)$$

where $g(n)$ is the cost already travelled and $h(n)$ is the estimated remaining cost.

The heuristic provides additional information about the goal compared with an uninformed method such as BFS.

How does the choice of heuristic affect A*?

The heuristic affects the order in which A* explores states.

For the warehouse problem, Manhattan distance is suitable because the robot moves only horizontally or vertically and every movement has a cost of 1.

In the experiments, Manhattan distance, $h(n) = 0$, Euclidean distance, and $2 \times \text{Manhattan}$ all produced a path length of 40 and 64 expanded states.

Therefore, no difference was observed in the measured results for this particular warehouse.

However, the heuristics have different theoretical properties. Manhattan distance is admissible for this grid, while $2 \times \text{Manhattan}$ can overestimate the remaining cost and is therefore not generally admissible.

What contribution did the LLM make?

The LLM was used as an engineering assistant to generate an implementation of A* based on a search-agent design that had already been specified.

It provided the implementation of the frontier, heuristic calculation, cost tracking, state expansion, goal detection, and path reconstruction.

The generated code was then tested and validated using multiple test cases rather than being accepted without verification.

What are the risks of using LLM-generated code without testing?

LLM-generated code may contain bugs, incorrect assumptions, or design choices that are unsuitable for a particular problem.

Code can appear correct while failing on edge cases such as an unsolvable map or a different input structure.

Therefore, testing is an important part of using an LLM as an engineering assistant.

The main lesson from this laboratory is that an LLM can assist with implementation, but the engineer remains responsible for understanding the problem, evaluating the generated code, testing its behaviour, and validating the final result.

12. Conclusion

The warehouse navigation problem was successfully formulated as a search problem and implemented using A* with LLM assistance.

The A* agent successfully solved the original warehouse with a path length of 40 moves and was able to handle one-step, unsolvable, and alternative-path test cases.

BFS and A* both found the same 40-move solution and expanded 64 states on the original warehouse. The heuristic investigation also produced identical measured results for Manhattan distance, $h(n)=0$, Euclidean distance, and $2 \times \text{Manhattan}$ on this particular map.

The laboratory demonstrated that LLMs can be useful for generating implementations, but generated code must be inspected and validated through systematic testing. The engineer remains responsible for the correctness and reliability of the final system.